infobip

Prompt Engineering

A Practical Skill for the AI Era

OCTOBER 2025

1

Table of contents

1. Introduction
2. Foundations
3. Prompt Engineering
4. Workshop

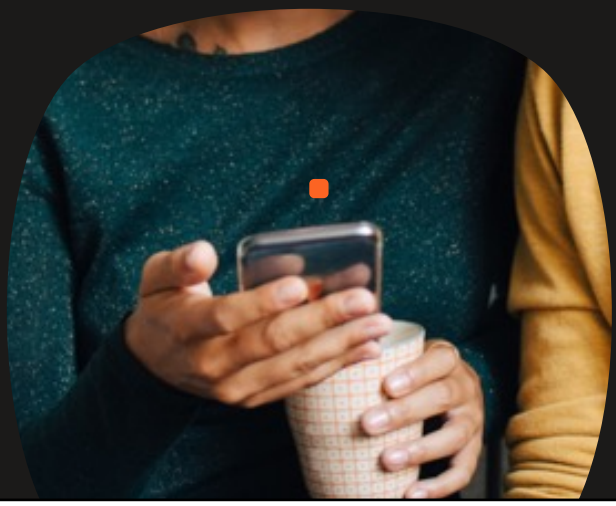
2

ONE SENTENCE QUOTE SLIDE

Life is like riding a bicycle. To keep your balance, you **must keep moving.**

3

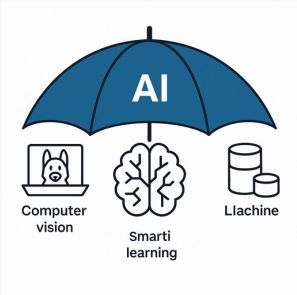
Foundations



4

AI Isn't New - But Something Changed 🤖

- **AI is an umbrella term that has existed for 70+ years**
- **Artificial Intelligence**: Strategies and techniques that enable machines to perform tasks that typically require human intelligence.



- **Pop Culture AI**:
  - ✅ Good: R2D2 (Star Wars), Robot (Lost in Space)
  - ⚠️ Bad: HAL 9000 (2001), Terminator, Skynet

5

Brief History - How We Got to 2025

1950s	1990s	2012	2017	2022	2025
Alan Turing asks "Can machines think? AI Winter - many promises, few results	**Expert Systems, chess computers (Deep Blue)** Algorithms exist, but lack data + compute	**AlexNet - Deep Learning breakthrough** ImageNet competition - AI beats humans!	**Transformer architecture - "Attention is All You Need"** Foundation for GPT, Claude, Gemini...	**ChatGPT publicly available** AI in millions of hands	**You are here!** 🎉 Next big breakthrough awaits after every corner

AI research has existed since the 1950s, but only NOW is it practically accessible to ordinary people. Why? Because of 3 converging factors...

6

Why Now? The Perfect Storm ⛈️

Key Formula: Same algorithms from the 2000s + 1000x more data + 1000x more compute power + Transformer architecture = ChatGPT, Claude, Gemini (2022+)

That's why AI is "suddenly" everywhere! Not suddenly - 70 years of research + 3 factors converged in the last 10 years.

01

💾 **DATA EXPLOSION** (Fuel for AI)

- 2000: Internet, email, websites
- 2010: Social media (Facebook, Twitter), smartphones
- 2020: Streaming, IoT, sensors everywhere

Result: Billions of images, texts, videos available for training

Pre-2010: Little data, expensive to collect

Post-2015: **Data is everywhere and cheap** (internet, cloud storage)

02

⚡ **COMPUTE POWER** (Processing Power)

- 2000: CPUs - slow for neural networks
- 2012: GPUs (gaming cards) - 10-100x faster for AI workloads
- 2020: Cloud computing (AWS, Azure, GCP) - rent compute by the hour

Result: Training models that used to cost millions $ now costs thousands $

- Pre-2010: Only tech giants could train large models
- Post-2015: **Cloud democratized compute** (anyone can rent GPUs)

03

🧠 **ALGORITHMIC BREAKTHROUGHS** (Smarter Models)

- 2000s: Neural networks exist, but shallow (1-3 layers)
- 2012: Deep Learning - we can train deep networks (50+ layers)
- 2017: Transformers - Attention mechanism = game changer

Result: Models that TRULY understand context and language

Pre-2017: Models good for narrow tasks (chess, image classification)

Post-2017: **Models generalize** - one model, many tasks

7

AI Hierarchy - Where Are We? 🎯

- **AI ⊃ ML ⊃ Gen AI ⊃ LLM**
  - **AI**: Any technology that mimics intelligence
  - **ML**: Systems that learn from data without explicit programming
  - **Gen AI**: Creates new content (text, images, code, audio)
  - **LLM**: Specialized for understanding and generating text
- **Foundation Models (The Foundation):**
  - Pre-trained models like GPT-4, Claude, Gemini, Llama
  - Trained on massive amounts of data
  - You **build on them** through prompting, don't train them yourself!
  - That's why prompt engineering is so important - it's how you use FMs
- **Today we're working with LLMs (which are Foundation Models for text)!**

```
┌──────────────────────────────────────┐
│   Artificial Intelligence (AI)       │  ← Broadest term
│  ┌──────────────────────────────┐    │
│  │   Machine Learning (ML)      │    │  ← Learning from data
│  │  ┌────────────────────────┐  │    │
│  │  │  Generative AI         │  │    │  ← Creates new content
│  │  │  ┌──────────────────┐  │  │    │
│  │  │  │ Large Language Models │  │  │  ← Specialized for text
│  │  │  │  (LLM)           │  │  │    │
│  │  │  │  • GPT-4, Claude  │  │  │    │
│  │  │  │  • Gemini, Llama  │  │  │    │
│  │  │  └──────────────────┘  │  │    │
│  │  └────────────────────────┘  │    │
│  └──────────────────────────────┘    │
└──────────────────────────────────────┘
```

8

How Do LLMs Work? 🧠

- **Transformer Architecture** (2017 - "Attention is All You Need")
- **Key Concept: ATTENTION MECHANISM**
- **Why is this revolutionary?**
  - ❌ Old approach: Read word-by-word (lose context)
  - ✅ Transformer: Looks at ENTIRE sentence at once (understands relationships)
- **Attention**: Model "pays attention" to relevant parts of text
- **Transformer explainer**: https://poloclub.github.io/transformer-explainer/

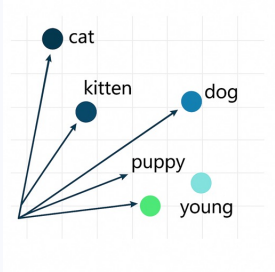
```
┌──────────────────────────────────────────┐
│ Input sequence                           │
│                                          │
│   "The cat sits on the ..."              │
└──────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────┐
│ Transformer model                        │
│                                          │
│ [ Embeddings ]                           │
│ [ Multi-Head Self-Attention ]  ×  N layers │
│ [ Feed Forward Network ]                 │
│ [ (positional info, etc.) ]              │
└──────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────┐
│ Next-token prediction                    │
│                                          │
│   "mat"   → 0.65 (65%)                   │
│   "table" → 0.30 (30%)                   │
│   "roof"  → 0.03 (3%)                    │
│   ...                                    │
└──────────────────────────────────────────┘
                    │
                    ▼
          Choose highest probability
                  "mat"
```

9

How LLMs Process Text Data 🔤

- 1. 🔪 **Tokenization** - Breaking text into pieces
  "The cat jumped" → ["The", "cat", "jump", "ed"]
- 2. 🔢 **Embeddings** - Converting to numbers
  "cat" → [0.2, 0.8, 0.3, 0.1, 0.5, ...]
  "kitten" → [0.25, 0.75, 0.35, 0.15, 0.48, ...]
  "puppy" → [0.6, 0.4, 0.5, 0.3, 0.7, ...]
- 3. 📍 **Vector Representations** - Similar meanings, similar positions

- **Key Insight**: Words with similar meanings have vectors that are "close" to each other in vector space.
- **Why this matters**: This is how models understand relationships



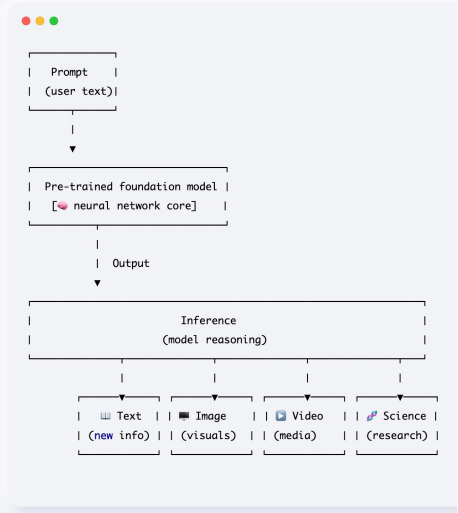
10

From Prompt to Output 💬

- **How LLMs Generate Text**:
  1. Input prompt: "Write a blog post about..."
  2. Tokenize → Embed → Process through Transformer layers
  3. Predict next token (word/subword)
  4. Add predicted token to sequence
  5. Repeat steps 3-4 until done
  6. 6. Output: "Artificial intelligence is transforming..."

Key Takeaway: LLMs generate text by predicting one token at a time, based on patterns learned from training data.

But there's a problem... What if the answer you need **ISN'T** in the training data? 🤔



11

Problem: LLMs Have Limited Knowledge 📚

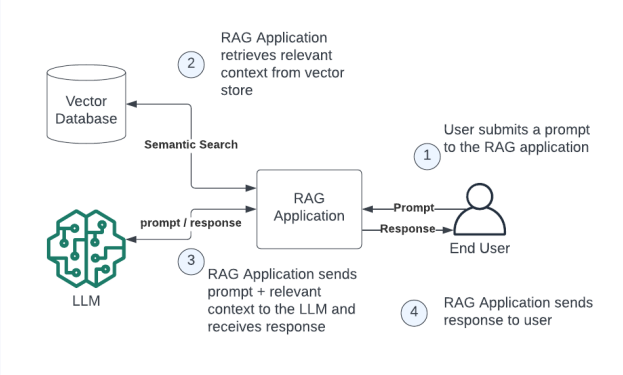
- **Challenge:**
  - Trained on data up to a certain date (knowledge cutoff)
  - No access to your internal documents
  - Don't know real-time information
- **Example**:
  - ❌ "What is TechCorp's Q3 2024 revenue?"
  - AI: "I don't have information about that specific company..."
  - ❌ "What are our HR policies for remote work?"
  - AI: "I don't know your internal policies..."

- **Solution?** → **Retrieval Augmented Generation (RAG)**

12

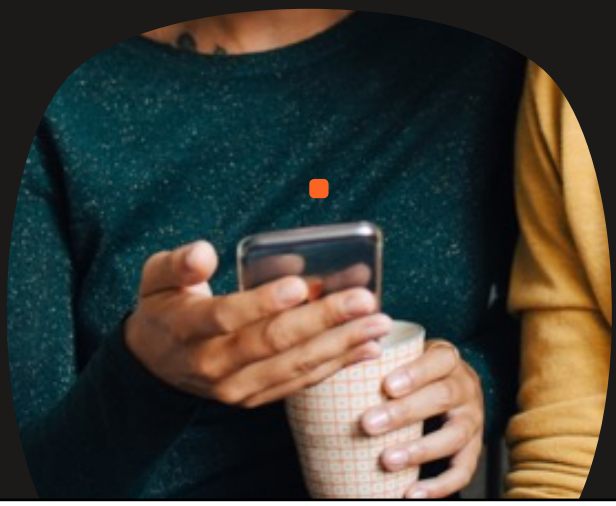
RAG - Retrieval Augmented Generation

- **Analogy: Open-book vs Closed-book exam**
  - **Closed-book** (classic LLM): Answers only from memory
  - **Open-book** (RAG): Can consult documents
- **Workflow**: [User Query] → [Vector DB Search] → [Retrieved Docs] → [LLM + Context] → [Answer]
- **Why is it powerful?**
  - ✅ Access to internal documents
  - ✅ Up-to-date information
  - ✅ Cites sources (fewer hallucinations)
  - ✅ Scalable (no need to retrain model)



13

Prompt Engineering



14

What is Prompt Engineering? ✍️

- **Definition**: The art and science of crafting prompts for AI models to produce optimal, precise, and useful responses.
- **Why is it important?**
  - 🎯 The difference between "ok" and "wow" results
  - 💰 More efficient API usage (fewer calls, better results)
  - 🎛️ Greater control over AI behavior
  - 🚀 Enables complex use cases
- **This is NOT:**
  - Just "writing prettier questions"
  - Experimenting without a system
  - Magic or a trick
- **This IS**:
  - Structured approach to communicating with AI
  - Understanding how models process queries
  - Applying proven techniques for better control
  - Iteratively improving prompts

15

Before & After - The Power of Good Prompts 💡

- ❌ Loš prompt:
  'Write an article about digital marketing.'"
- Što dobijete? Generički, površan content. Pretjerano dug ili prekratak. Pun fluff-a: 'Of course, I'd be happy to help you...
- ✅ Dobar prompt:
  You are a senior digital marketing strategist writing for B2B SaaS decision-makers. Write a 500-word blog post on the top 3 digital marketing trends for B2B SaaS companies in 2025. Include specific statistics, practical implementation tips, and 1-2 real company examples. Format: Use H2 headings for each trend. Tone: Professional but conversational.
- Što dobijete? Fokusiran, actionable content. Precizan format. Relevantna dubina. Direktan odgovor bez bullshit-a.

16

Prompt Engineering Techniques - Overview 🎯

1. 📚 Learning from Examples

- Show the model what you want through examples
  - **Zero-shot**: No examples, just task description
  - **Few-shot**: Give 2-5 examples to guide the model
- **Best for**: Specific formats, styles, domain-specific language

2. 🎯 **Being Directive**

- Tell the model HOW to do the task
  - **Specific instructions**: "Write 200 words...", "Use JSON format..."
  - **Constraints**: "Don't include...", "Only focus on..."
- **Best for**: When you need exact specifications

3. 🧠 Guiding Reasoning

- Help the model think step-by-step
  - **Chain of Thought (CoT)**: Break problems into logical steps
  - **"Think step by step"**: Ask model to show its reasoning
- **Best for**: Complex problems, math, planning, debugging

4. ✨ Refining Results

- Improve outputs through iteration
  - **Iterative refinement**: Test, adjust, repeat
  - **Human review**: Quality check before use
- **Best for**: Production applications, high-stakes content

Think of prompting techniques as tools in a toolbox.
Prompt engineering is NOT trial-and-error. It's the systematic application of proven techniques.

17

Core Prompt Engineering Techniques 🛠️

Today you'll learn

1. **Basic Structure** - How to structure a prompt
2. **Clarity & Directness** - Clarity and specificity
3. **Role Prompting** - Assigning roles to AI
4. **Data Separation** - Separating instructions from data (XML tags)
5. **Output Formatting** - Controlling output format (JSON, XML)
6. **Chain of Thought** - "Think step-by-step"
7. **Few-Shot Learning** - Learning from examples

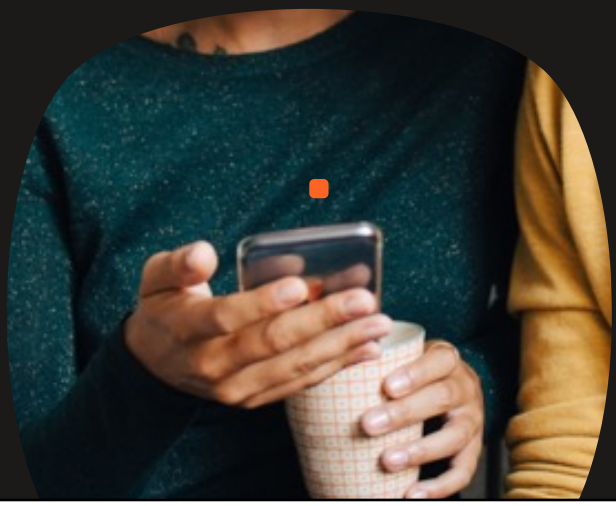
Foundation: Anthropic's Prompt Engineering Tutorial

ANTHROP\C

18

Workshop

Anthropic's Prompt Engineering Interactive Tutorial



19

Practical Use Cases 🎯

Developers:
- 🐛 Code generation and debugging
- 📝 Writing documentation
- 🔍 Code review assistants
- 🤖 Building AI-powered features

Product Managers:
- 💡 Feature ideation and brainstorming
- 📊 User feedback analysis
- 📋 Writing PRDs and specs
- 🎨 Creating mockup descriptions

Product Managers:
- 💡 Feature ideation and brainstorming
- 📊 User feedback analysis
- 📋 Writing PRDs and specs
- 🎨 Creating mockup descriptions

Content Creators:
- ✍️ Blog posts, social media content
- 📧 Email campaigns
- 🎬 Video scripts
- 🔄 Content repurposing

20

Workshop Logistics 📋

- **Timeline**
  - This presentation ✅ [30 min]
  - Setup check & API key distribution [5 min]
  - Exercises 1-2 (Basic + Clarity) [20 min]
  - Break [5 min]
  - Exercises 3-7 (Roles → Few-shot) [20 min]
  - Wrap-up, Q&A, Next steps

21

Quick Setup Check ⚙️

Track 1: Jupyter

```
# Terminal check:
python3 --version  # Should show 3.8+
git --version      # Should show Git installed

# Clone repo:
cd ~/Desktop
git clone https://github.com/anthropics/prompt-eng-interactive-tutorial.git
cd prompt-eng-interactive-tutorial/Anthropic\1P
jupyter notebook
```

Track 2: Free-Tier

- Go to: claude.ai, chatgpt.com, or gemini.google.com
- Login or create account
- Open new chat

Help each other! 🤝

22

Inference Parameters - Controlling AI Behavior 🎛️

- **What are inference parameters?**
  - Settings that control how AI generates responses - from completely predictable to creative.
- **Key Parameters**:
  - 🌡️ Temperature (0.0 - 1.0)
    - 0.0 🤖 Robot (same every time)
    - 1.0 Artist 🎨 (always different)
- **When to use**:
  - **Temperature = 0**: Code generation, data extraction, JSON output
  - **Temperature = 0.5-0.7**: Balanced responses, general Q&A
  - **Temperature = 0.8-1.0:** Creative writing, brainstorming, marketing
- 📊 **Top-P / Nucleus Sampling (0.0 - 1.0)**
  - Controls diversity of word choices
  - **0.9 (default):** Considers top 90% probable words
  - Combines with temperature for fine-tuning
- 📏 **Max Tokens**
  - Limits response length
  - **Important for**: Cost control, consistency
  - **Example:** Tweet = 50 tokens, Blog post = 1000 tokens

23

Q&A

24